TransitOps Deployment Guide

Deployment Guide

TransitOps — Fleet & Logistics Operations Platform

Version: 1.0 **Companion documents:** TRD, README, Backend Schema

---

1. Deployment Options

Option	Use Case
Docker Compose	Local development, demos, single-node small-scale deployment
Kubernetes	Multi-replica, cluster-managed production deployment
Bare-metal / VM + PM2	Deployment without containers

---

2. Option A: Docker Compose (recommended for demo/hackathon judging)

2.1 Architecture

Three services on a shared bridge network (`transitops_network`): - **`api`** — Node.js/Express backend, built from the root `Dockerfile`, port `5000` - **`mongo`** — official `mongo:6.0` image, data persisted to named volume `mongo_data`, port `27017` - **`nginx`** — `nginx:alpine`, serves `./frontend` statically and reverse-proxies `/api/` to `api:5000`, port `80`

2.2 Steps

```
cd transitops
docker compose up --build
```

This builds the API image (multi-stage: installs prod deps in a builder stage, then copies into a slim `node:18-alpine` runtime image running under `pm2-runtime -i max`), starts MongoDB, and starts Nginx.

2.3 Verify

```
curl http://localhost:5000/api/v1/health
# Frontend via Nginx:
open http://localhost
```

2.4 Seed Demo Data

```
docker exec -it transitops_api node backend/seed.js
```

2.5 Configuration Note

`docker-compose.yml` currently hardcodes `JWT_SECRET=transitops_super_secret_key_2026` and a default `MONGO_URI` directly in the `environment:` block. **For anything beyond local demo use, replace this with a `.env` file referenced via `env_file:`, or Docker secrets** — do not ship the hardcoded secret to any shared or public environment.

2.6 Stopping / Cleanup

```
docker compose down          # stop containers, keep volume (data persists)
docker compose down -v       # stop containers and delete the mongo_data volume
```

---

3. Option B: Kubernetes

3.1 Manifests Provided

- `k8s/deployment.yaml` — Deployment `transitops-api`, 3 replicas, image `transitops/api:latest`, container port `5000`
- `k8s/service.yaml` — ClusterIP Service `transitops-api-service`, port `80` → target port `5000`

3.2 Resource Requests/Limits (as defined)

	CPU	Memory
Request	250m	512Mi
Limit	500m	1Gi

3.3 Health Probes (as defined)

Probe	Path	Initial Delay	Period
Liveness	`/api/v1/health`	15s	20s
Readiness	`/api/v1/health`	5s	10s

3.4 ⚠ Missing Manifests — Required Before Applying

The Deployment references a ConfigMap (`transitops-config`) and a Secret (`transitops-secrets`) via `envFrom`, but **neither manifest exists in the repository**. You must create both before `kubectl apply` will produce a healthy pod. Example:

```
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: transitops-config
data:
  PORT: "5000"
  NODE_ENV: "production"
  MONGO_URI: "mongodb://<your-mongo-host>:27017/transitops"
```

```
# secret.yaml (do not commit real values)
apiVersion: v1
kind: Secret
metadata:
  name: transitops-secrets
type: Opaque
stringData:
  JWT_SECRET: "<generate-a-strong-secret>"
  ENCRYPTION_SECRET: "<generate-a-strong-secret>"
```

3.5 Build & Push the Image

```
docker build -t <your-registry>/transitops-api:latest .
docker push <your-registry>/transitops-api:latest
# then update image: in k8s/deployment.yaml to match
```

3.6 Apply

```
kubectl apply -f configmap.yaml
```

```
kubectl apply -f secret.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods -w
```

3.7 Expose Externally

`transitops-api-service` is `ClusterIP` only (internal). Add an `Ingress` resource or change the Service `type` to `LoadBalancer` / `NodePort` to expose it — neither is currently defined in the repo. The frontend also has no Kubernetes manifest; it would need its own static-hosting Deployment/Service (or be served via the same Ingress from a bucket/CDN).

3.8 Known Gap: Multi-Replica Cache Consistency

The API uses an in-process `node-cache` for GET response caching. With `replicas: 3`, a write to one pod does not invalidate cached GET responses held by the other two pods — clients may see stale data for up to the 300s TTL depending on which pod they hit. Acceptable for a hackathon deployment; for real multi-replica production use, migrate the cache to Redis (see TRD §6).

---

4. Option C: Bare-Metal / VM with PM2

```
cd transitops/backend
npm ci --only=production
npm install -g pm2
cp .env.example .env   # fill in real values
pm2 start ../ecosystem.config.js --env production
pm2 save
pm2 startup            # configure PM2 to launch on boot
```

`ecosystem.config.js` runs the app in cluster mode across all available CPU cores (`instances: 'max'`), auto-restarts on crash, restarts if memory exceeds 1GB, and logs to `logs/pm2-error.log` / `logs/pm2-out.log`.

Serve the `frontend/` directory with any static web server (Nginx, Caddy, `serve`) and reverse-proxy `/api/` to the PM2-managed Node process, following the same pattern as `nginx/nginx.conf`.

---

5. Environment Variables Checklist (all deployment options)

Variable	Required	Notes
`MONGO_URI`	✅	Point at a real, reachable MongoDB instance
`JWT_SECRET`	✅	Server exits on boot if unset — generate a strong random value per environment
`ENCRYPTION_SECRET`	Strongly recommended	Falls back to an insecure hardcoded value in `models/Driver.js` if unset
`NODE_ENV`	Recommended	Set `production` to suppress stack traces in error responses
`FRONTEND_URL`	Recommended in prod	Restricts CORS to your real frontend origin
`PORT`	Optional	Defaults to `5000`
`JWT_EXPIRE`	Optional	Defaults to `7d`
`SMTP_HOST` / `SMTP_USER` / `SMTP_PASS`	Optional	Without these, password-reset emails go to an Ethereal test inbox (not delivered to real users)

---

6. Post-Deployment Smoke Test

1. `GET /api/v1/health` → 200
2. `POST /api/v1/auth/login` with a seeded demo account → 200 + token
3. `GET /api/v1/vehicles` with that token → 200
4. `GET /api-docs` → Swagger UI loads
5. Frontend loads at the Nginx/Ingress URL and can log in

7. Rollback

Docker Compose: `docker compose down`, redeploy previous image tag or `git checkout` the previous commit and `docker compose up --build`. **Kubernetes:** `kubectl rollout undo deployment/transitops-api`. **PM2:** `pm2 reload ecosystem.config.js --env production` after checking out the previous release; PM2's cluster reload is zero-downtime.

Companion documents: TRD (deployment topology detail), README (local setup), Backend Schema, SRS.